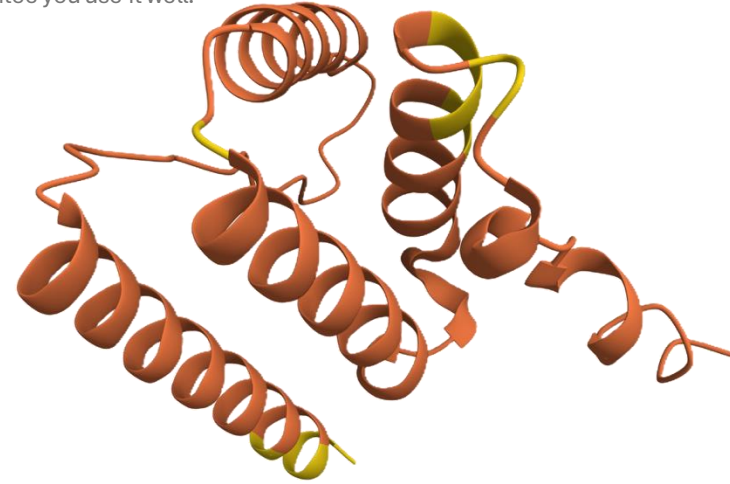




Alphafold 2



Alphafold 3

AlphaFold on Google TPUs

How fast can protein folding scale?

Lorenzo Pazienza & Ihab El Bani

Stanford Summer Session 2026

Introduction to High Performance Computing and AI Systems

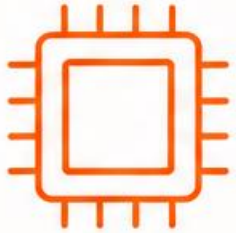
13 Aug 2026

<https://github.com/lorenzopazienza/alphafold-tpu-benchmark>

<https://alphafold-tpu.vercel.app>



Different hardware | Huge differences

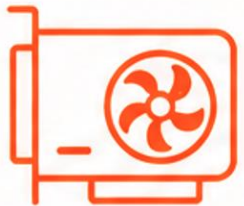


CPU • Intel Xeon (2 vCPU)

212 s



Baseline



GPU • NVIDIA Tesla T4

13.1 s



16.2x
faster than CPU



TPU • TPU v5e Lite

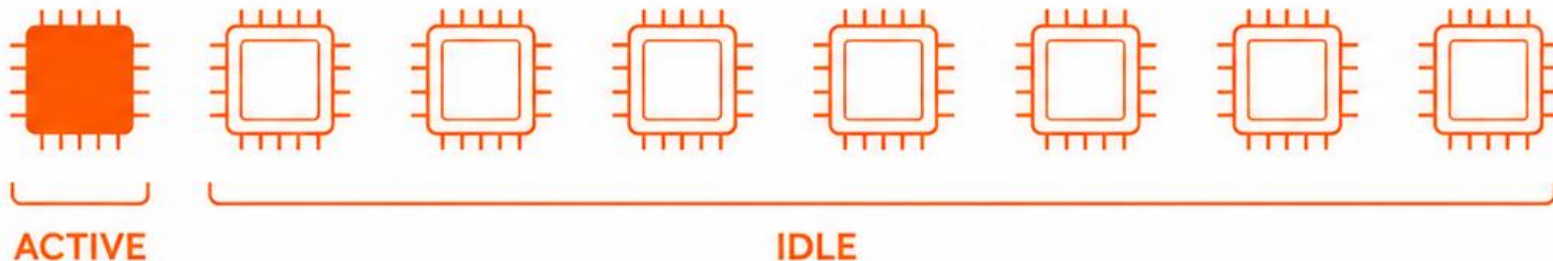
0.47 s |

451x
faster than CPU
27x
faster than GPU



But the TPU was waiting

1 / 8 chips
actually doing the work



Cold start

27 s



Steady state

0.47 s

FIRST PREDICTION



~76% of first prediction → JAX/XLA compilation

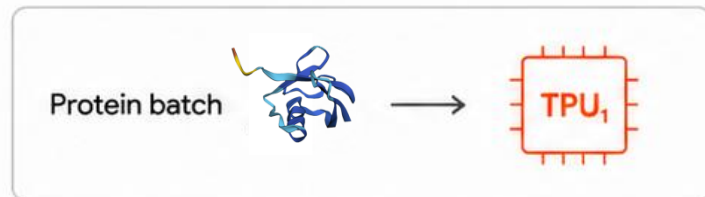
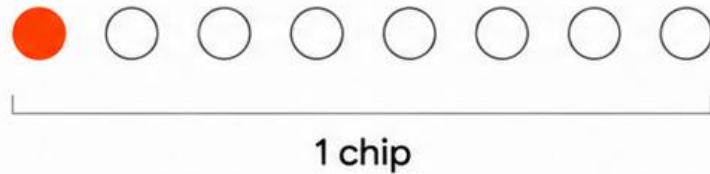


From vectorization to parallelism

Built on Docker + Kubernetes (GKE, Kueue-managed TPU quota); JAX/XLA compilation throughout.

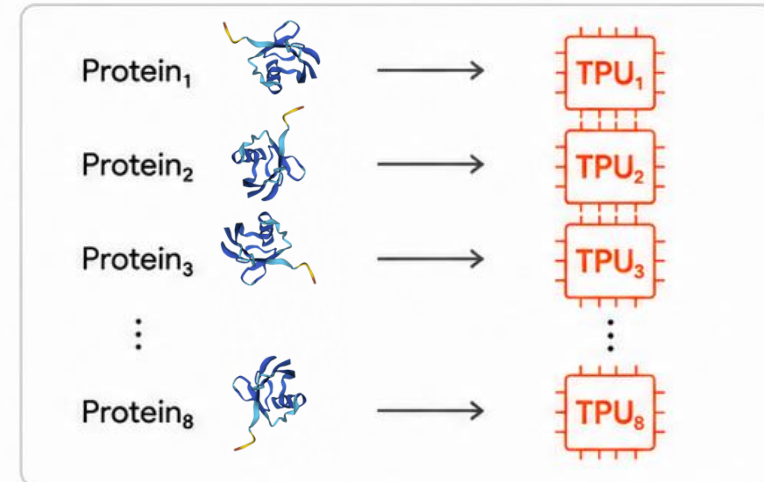
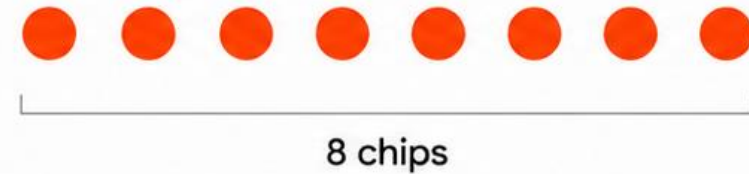
vmap

Vectorizes $\neq$ distributes



pmap

Real multi-chip execution



6.92x
throughput
8 TPU chips utilized



The bottleneck wasn't the hardware. **It was how we exposed parallelism to the hardware.**



Near-linear scaling

$$\text{Throughput} \propto \text{chips}^{0.963}$$

$$R^2 = 0.981$$

Flat price

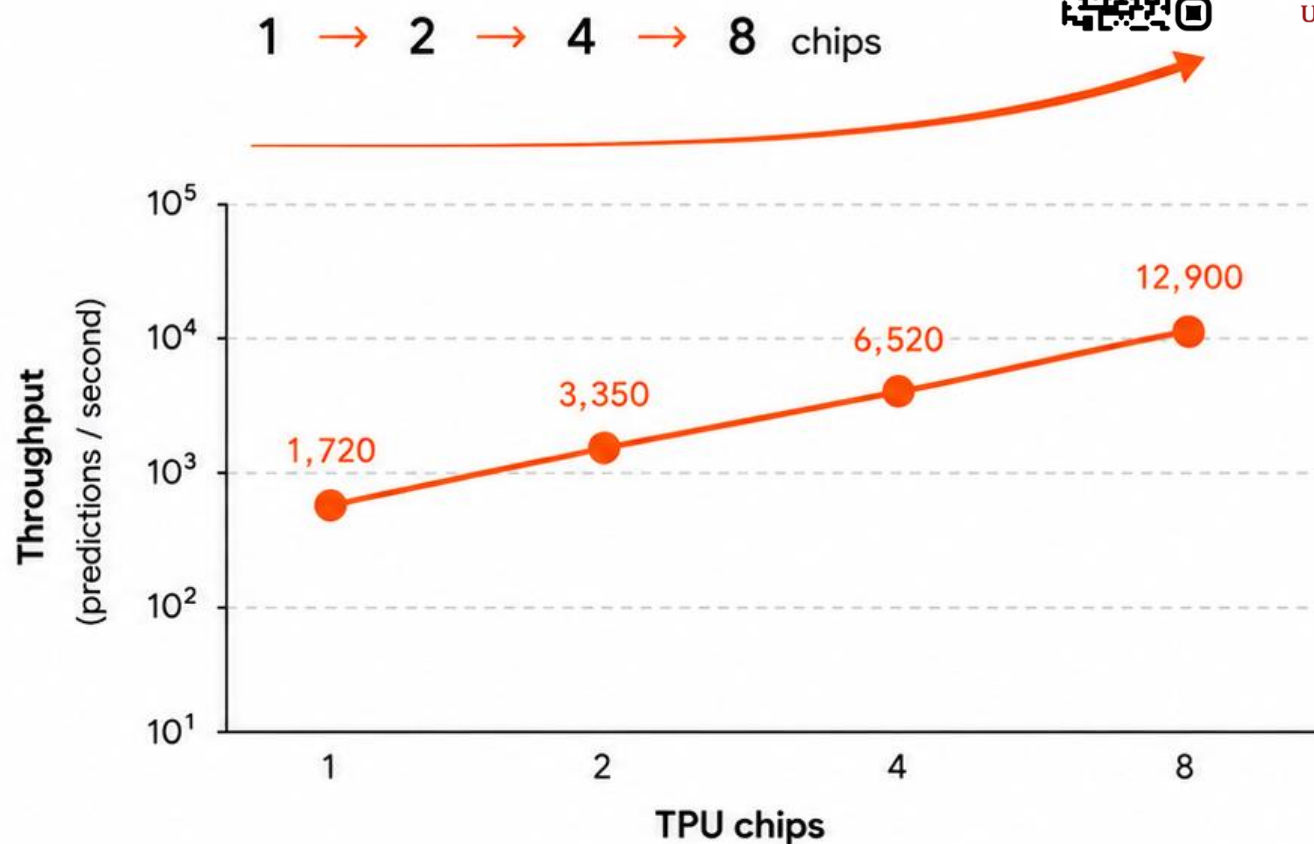
CPU	GPU	TPU
\$11.19	\$1.27	\$1.25



Doubling TPU chips $\approx$
doubling throughput.



Our main takeaway is simple: specialized hardware gives you the potential for acceleration but software parallelism determines how much of that potential you actually use.



$$\text{Throughput} \approx 4527.77 \times \text{chips}^{0.963} \times \text{length}^{-1.572}$$